Tunnijuhend: lisamine ja kustutamine

11IT · Veebirakendused · 9. tund · tunnis

Täna õpetad serveri arvustusi vastu võtma ja kustutama ning ühendad need lehega.

> **Vaja läheb:** Node.js, npm, VS Code ja söökla projekt või kursuse lehelt algfailid.
>
> **Kuidas töötada:** ennusta, mis juhtub → kirjuta kood → käivita → selgita, mis muutus.
>
> **Kaks reeglit.** Iga kord, kui muudad faili `server/index.js`, pane server `Ctrl+C`-ga kinni ja käivita uuesti `node index.js`, sest Node ei loe faili ise uuesti. Serveri vead on serveri terminalis, lehe vead on konsoolis (`F12`).
>
> **Miinimum:** sammud 0–3, kontrollitud arvustuse lisamine. Sammud 4 ja 5 on kustutamine.

Samm 0: mõlemad serverid tööle

Ava söökla projekt. **Kui sul projekti pole**, laadi alla `tunnitoo-lisamine.zip`, paki lahti, ava kaust `sookla` ja tee terminalis `npm install`, siis `cd server`, `npm install` ja `cd ..`.

Esimeses terminalis käivita server, teises leht. Uue terminali saad nupuga **+**. Mõlema terminali alguses kontrolli `pwd`-ga, et oled projekti peakaustas, kus on `vite.config.ts`.

`esimene terminal`

```
cd server
node index.js
```

`teine terminal`

```
npm run dev
```

Nüüd peaksid nägema: ava aadress, mille Vite terminali kirjutas, tavaliselt `http://localhost:5173`: lehel on kolm arvustust.

Kui näed midagi muud: kui lehel on „Arvustusi ei saanud kätte“, kontrolli esmalt, kas server töötab. Kui `npm run dev` annab vea, oled tõenäoliselt vales kaustas: `cd ..` viib tagasi.

Samm 1: valmista server ette

Ava `server/index.js`. Tee kaks väikest muudatust. Esiteks lisa kohe rea `const app = express();` järele:

```
app.use(express.json());
```

See parsib päringu kehas tuleva JSON-i, nii et saame seda lugeda kujul `req.body`. Rida peab olema enne marsruute.

Teiseks muuda `const arvustused` sõnaks `let arvustused` ja lisa massiivi lõpu järele uus rida:

```
let jargmineId = 4;
```

`let` lubab muutujale hiljem uue väärtuse anda. `jargmineId` on number, mille saab järgmine uus arvustus. Neli, sest numbrid 1–3 on juba kasutusel.

Samm 2: POST-marsruut

Lisa see kood olemasoleva `app.get('/api/arvustused', ...` marsruudi järele:

```
app.post('/api/arvustused', (req, res) => {
  const { toit, hinne, kommentaar } = req.body;

  const toitOnKorras = typeof toit === 'string' && toit.trim() !== '';
  const kommentaarOnKorras = typeof kommentaar === 'string' && kommentaar.trim() !== '';
  const hinneOnKorras = Number.isInteger(hinne) && hinne >= 1 && hinne <= 5;

  if (!toitOnKorras || !kommentaarOnKorras || !hinneOnKorras) {
    res.status(400).json({ viga: 'Täida kõik lahtrid, hinne on 1 kuni 5.' });
    return;
  }

  const uus = { id: jargmineId, toit: toit.trim(), hinne, kommentaar: kommentaar.trim() };
  jargmineId = jargmineId + 1;
  arvustused = [uus, ...arvustused];
  res.status(201).json(uus);
});
```

Kood	Mida see teeb
`app.post('/api/arvustused',`	See reegel kehtib POST-päringutele sellel aadressil.
`(req, res) => {`	Funktsioon saab päringu (req) ja vastuse (res).
`const { toit, hinne, kommentaar } = req.body;`	Võtab päringu kehast kolm välja.
`typeof toit === 'string' && toit.trim() !== ''`	toit peab olema tekst ja selles peab olema muud kui tühikud. && on „ja“.
`Number.isInteger(hinne) && ...`	hinne peab olema täisarv 1 kuni 5.
`if (!toitOnKorras || ...`	! on „ei“, || on „või“. Kui midagi pole korras.
`res.status(400).json({ viga: ... });`	Vastab koodiga 400 ja veateatega.
`return;`	Lõpetab funktsiooni, nii et edasi ei minda.
`const uus = { id: jargmineId, ... };`	Teeb uue arvustuse. Numbri annab server, trim() eemaldab tühikud.
`jargmineId = jargmineId + 1;`	Järgmine saab ühe võrra suurema numbri.
`arvustused = [uus, ...arvustused];`	Uus massiiv: ees uus arvustus, kolm punkti laotavad vanad järele.
`res.status(201).json(uus);`	Vastab koodiga 201 ja saadab uue arvustuse tagasi.

Salvesta. Serveri terminalis: `Ctrl+C`, siis `node index.js`.

Nüüd proovi päringut ilma Reactita. Ava oma leht, vajuta `F12`, vali vahekaart **Console** ja kirjuta sinna:

```
const vastus = await fetch('/api/arvustused', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ toit: 'Test', hinne: 5, kommentaar: 'Hea' }),
});
await vastus.json();
```

Mis sinu arvates tagasi tuleb? Vajuta Enter.

Nüüd peaksid nägema: konsoolis objekt koos `id`-ga. Kui lisasid esimese arvustuse pärast serveri käivitamist, on `id` 4. Ava `http://localhost:3000/api/arvustused`: test-arvustus on nimekirja alguses. Kui konsool ütleb, et kleepimine pole lubatud, kirjuta käsk `allow pasting` ja proovi uuesti.

Kui näed midagi muud: 500 ja `Cannot destructure` terminalis tähendab, et `req.body` on `undefined`: kontrolli rida `app.use(express.json());`. 404 tähendab sageli, et serverit pole uuesti käivitatud.

Samm 3: pane vorm serverile saatma

Ava `src/App.tsx` ja asenda kogu funktsioon `lisa` selle koodiga:

```
  async function lisa(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setViga('');

    try {
      const vastus = await fetch('/api/arvustused', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ toit, hinne, kommentaar }),
      });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }

      setArvustused((praegused) => [andmed, ...praegused]);
      setToit('');
      setHinne(5);
      setKommentaar('');
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }
```

Kood	Mida see teeb
`async function lisa(e: ...) {`	Nüüd on see async, sest kasutame sees await-i.
`e.preventDefault();`	Muidu teeks brauser vormi saatmisel lehele värskenduse.
`setViga('');`	Kustutab eelmise veateate.
`try {`	Kui server ei vasta üldse, hüppab programm catch-i.
`await fetch('/api/arvustused', {`	Teine argument on objekt päringu seadetega.
`method: 'POST',`	Meetod.
`headers: { 'Content-Type': 'application/json' },`	Päringu päis ütleb serverile, et keha on JSON.
`body: JSON.stringify({ ... }),`	Teeb objektist teksti. toit on lühend kirjutisest toit: toit.
`const andmed = await vastus.json();`	Loeb vastuse: uus arvustus või { viga: ... }.
`if (!vastus.ok) {`	Kui server vastas veakoodiga.

Kood	Mida see teeb
`setViga(andmed.viga);`	Näitab serveri veateadet. Ekraanil näitab seda eelmises tunnis lisatud rida.
`setArvustused((praegused) => [andmed, ...praegused]);`	Paneb serveri antud arvustuse nimekirja algusesse. praegused on oleku kõige värskem väärtus.
`} catch {`	Siia jõuab programm, kui server ei vasta või vastus pole JSON.

Mis sinu arvates juhtub, kui lisad arvustuse ja vajutad siis `F5`? Proovi.

Nüüd peaksid nägema: arvustus ilmub nimekirja ja pärast `F5`-e **on see ikka alles!** Vajuta nüüd „Lisa“ tühja vormiga: lehel on teade „Täida kõik lahtrid, hinne on 1 kuni 5.“ See teade tuli serverist. **Miks** on hea, et kontroll on serveris?

Kui näed midagi muud: kui lehel on „Serveriga ei saanud ühendust“, kontrolli, kas server töötab ja kas fetch-i aadress on õige.

Samm 4: DELETE-marsruut

Lisa faili `server/index.js` POST-marsruudi järele:

```
app.delete('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const leitud = arvustused.find((a) => a.id === id);

  if (!leitud) {
    res.status(404).json({ viga: 'Sellist arvustust pole.' });
    return;
  }

  arvustused = arvustused.filter((a) => a.id !== id);
  res.json(leitud);
});
```

Kood	Mida see teeb
`app.delete('/api/arvustused/:id',`	:id on muutuv osa aadressis.
`Number(req.params.id)`	Aadressist saadud tekst "2" muutub arvuks 2.
`arvustused.find((a) => a.id === id)`	.find() otsib esimese sobiva. === on „täpselt võrdne“.
`if (!leitud) {`	Ei leidnud: vastab 404 ja lõpetab.
`arvustused.filter((a) => a.id !== id)`	.filter() teeb uue massiivi. !== on „ei ole võrdne“.
`res.json(leitud);`	Vastab kustutatud arvustusega. Staatus on vaikimisi 200.

Salvesta ja käivita server uuesti.

Samm 5: kustuta-nupp

Kõigepealt anname kaardile nupu. Muuda fail `src/ArvustuseKaart.tsx` selliseks:

`src/ArvustuseKaart.tsx`

```
import type { Arvustus } from './types';

type Props = {
  arvustus: Arvustus;
  kustuta: (id: number) => void;
};

export default function ArvustuseKaart({ arvustus, kustuta }: Props) {
  const tahed = '★'.repeat(arvustus.hinne) + '☆'.repeat(5 - arvustus.hinne);
  return (
    <li className="kaart">
      <div className="kaart-pea">
        <strong>{arvustus.toit}</strong>
        <span className="tahed">{tahed}</span>
      </div>
      <p>{arvustus.kommentaar}</p>
      <button type="button" onClick={() => kustuta(arvustus.id)}>Kustuta</button>
    </li>
  );
}
```

Kood	Mida see teeb
`kustuta: (id: number) => void;`	Uus prop: funktsioon, mis saab arvu ega tagasta midagi (void).
`({ arvustus, kustuta }: Props)`	Võtame propsidest välja ka kustuta.
`onClick={() => kustuta(arvustus.id)}`	Nupuvajutusel kutsu kustuta selle arvustuse numbriga. Noolefunktsioon on vajalik, muidu kutsutaks see välja juba joonistamise ajal.

VS Code joonib nüüd App-is rea `<ArvustuseKaart` punaseks. TypeScript annab märku, et vajalik omadus `kustuta` on puudu. Lisa `App.tsx`-i funktsiooni `lisa` järele:

```
  async function kustuta(id: number) {
    setViga('');

    try {
      const vastus = await fetch(`/api/arvustused/${id}`, { method: 'DELETE' });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }

      setArvustused((praegused) => praegused.filter((a) => a.id !== id));
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }
```

Kood	Mida see teeb
`fetch(`/api/arvustused/${id}`,`	Langetusrõhumärgid ehk tagurpidi ülakomad: ${id} asemele tuleb number.
`{ method: 'DELETE' }`	Keha pole vaja, kõik on aadressis.

Kood	Mida see teeb
`setArvustused((praegused) => praegused.filter(...));`	Eemaldab ka lehelt, nii et ei pea kõike uuesti küsima.

Ja anna funktsioon kaardile:

```
<ArvustuseKaart key={a.id} arvustus={a} kustuta={kustuta} />
```

Nüüd peaksid nägema: igal kaardil on punakas nupp „Kustuta“. Vajuta seda: kaart kaob. Vajuta `F5`: see on ikka kadunud.

Kui näed midagi muud: kui arvustused kaovad ilma nupuvajutuseta, on `onClick`-is puudu `() =>`.

Samm 6: taaskäivita server

Lisa arvustus ja kustuta üks vana. **Mis sinu arvates juhtub**, kui serveri taaskäivitad? Pane server `Ctrl+C`-ga kinni, käivita uuesti ja värskenda lehte.

Nüüd peaksid nägema: alles on jälle algsed kolm arvustust. Massiiv elab programmi mälus ja programmi sulgedes see kaob. **Miks see juhtus?** Andmete säilimiseks tuleb need püsivalt salvestada, edaspidi kasutame selleks andmebaasi.

Samm 7: salvesta Gitti

Ava terminal, kus ükski server ei tööta, kontrolli, et oled projekti peakaustas, ja tee:

```
git add .
git commit -m "Arvustuste lisamine ja kustutamine"
```

Kui Git ütleb `not a git repository`, on projekt ZIP-failist: tee enne seda üks kord `git init`. Kui Git küsib nime ja e-posti, täida need käskudega `git config --global user.name "Sinu Nimi"` ja `git config --global user.email "sinu@naide.ee"`.

Kogu fail

Kui midagi ei tööta, võrdle oma faile nendega.

server/index.js

```
const express = require('express');

const app = express();

app.use(express.json());

let arvustused = [
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },
];
let jargmineId = 4;

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});

app.post('/api/arvustused', (req, res) => {
  const { toit, hinne, kommentaar } = req.body;

  const toitOnKorras = typeof toit === 'string' && toit.trim() !== '';
  const kommentaarOnKorras = typeof kommentaar === 'string' && kommentaar.trim() !== '';
  const hinneOnKorras = Number.isInteger(hinne) && hinne >= 1 && hinne <= 5;

  if (!toitOnKorras || !kommentaarOnKorras || !hinneOnKorras) {
    res.status(400).json({ viga: 'Täida kõik lahtrid, hinne on 1 kuni 5.' });
    return;
  }

  const uus = { id: jargmineId, toit: toit.trim(), hinne, kommentaar: kommentaar.trim() };
  jargmineId = jargmineId + 1;
  arvustused = [uus, ...arvustused];
  res.status(201).json(uus);
});

app.delete('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const leitud = arvustused.find((a) => a.id === id);

  if (!leitud) {
    res.status(404).json({ viga: 'Sellist arvustust pole.' });
    return;
  }

  arvustused = arvustused.filter((a) => a.id !== id);
  res.json(leitud);
});

app.get('/api/menyy', (req, res) => {
  res.json(['Hernesupp', 'Kanakaste riisiga', 'Pannkoogid']);
});

app.listen(3000, () => {
  console.log('Server töötab: http://localhost:3000');
});
```

src/App.tsx

```
import { useEffect, useState } from 'react';
import type { FormEvent } from 'react';
import type { Arvustus } from './types';
import ArvustuseKaart from './ArvustuseKaart';
import './App.css';

export default function App() {
  const [arvustused, setArvustused] = useState<Arvustus[]>([]);
  const [laeb, setLaeb] = useState(true);
  const [viga, setViga] = useState('');
  const [toit, setToit] = useState('');
  const [hinne, setHinne] = useState(5);
  const [kommentaar, setKommentaar] = useState('');

  async function laeArvustused() {
    try {
      const vastus = await fetch('/api/arvustused');
      if (!vastus.ok) {
        throw new Error('Server vastas veaga');
      }
      const andmed: Arvustus[] = await vastus.json();
      setArvustused(andmed);
    } catch {
      setViga('Arvustusi ei saanud kätte. Kas server töötab?');
    }
    setLaeb(false);
  }

  useEffect(() => {
    laeArvustused();
  }, []);

  async function lisa(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setViga('');

    try {
      const vastus = await fetch('/api/arvustused', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ toit, hinne, kommentaar }),
      });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }
```

```
      setArvustused((praegused) => [andmed, ...praegused]);
      setToit('');
      setHinne(5);
      setKommentaar('');
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }

  async function kustuta(id: number) {
    setViga('');

    try {
      const vastus = await fetch(`/api/arvustused/${id}`, { method: 'DELETE' });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }

      setArvustused((praegused) => praegused.filter((a) => a.id !== id));
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }

  return (
    <main>
      <h1>Söökla arvustused</h1>

      <form onSubmit={lisa}>
        <input value={toit} onChange={(e) => setToit(e.target.value)} placeholder="Mis toit?" />
        <select value={hinne} onChange={(e) => setHinne(Number(e.target.value))}>
          <option value={5}>5 ★</option>
          <option value={4}>4 ★</option>
          <option value={3}>3 ★</option>
          <option value={2}>2 ★</option>
          <option value={1}>1 ★</option>
        </select>
        <input
          value={kommentaar}
          onChange={(e) => setKommentaar(e.target.value)}
          placeholder="Kommentaar"
        />
        <button type="submit">Lisa</button>
      </form>

      {laeb && <p className="kokku">Laen arvustusi…</p>}
      {viga && <p className="kokku">{viga}</p>}

      <ul>
        {arvustused.map((a) => (
          <ArvustuseKaart key={a.id} arvustus={a} kustuta={kustuta} />
        ))}
      </ul>
    </main>
  );
}
```

Kui jõuad varem valmis

Valikulised väljakutsed. Koodi pole, ainult vihjed.

- **Muutmine.** Marsruut `app.put('/api/arvustused/:id', ...)`, mis muudab olemasolevat arvustust. Vihje: `.map()` ja sama kontroll, mis POST-is.
- **Üks arvustus.** `GET /api/arvustused/2` annab ühe arvustuse või 404. Vihje: `.find()`.
- **Liiga pikk kommentaar.** Vasta 400, kui kommentaar on pikem kui 200 märki. Vihje: `kommentaar.length`.